Chapter 04: Spring MVC

Department of Software Engineering
Faculty of Information Technology (HCM-UTE)
Course ID: WEPR330479
Lecturer: Msc. Khoa Hoang

Content

- What is Spring MVC?
- Architecture of Spring MVC
- Thymeleaf template engine
- Demo

What is Spring MVC?

What is Spring MVC?

- Spring Web MVC is the original web framework built on the Servlet API and has been included in the Spring Framework from the very beginning.
- The Spring Web MVC framework is designed around the Model-View-Controller design pattern to help developers build flexible, loosely coupled web applications and REST APIs.
- The default handler is based on the @Controller and @RequestMapping annotations, offering a wide range of flexible handling methods.
- Parallel to Spring Web MVC, Spring Framework 5.0 introduced a reactive-stack web framework called Spring WebFlux.

Documentation

- Official documentation: https://docs.spring.io/spring-framework/reference/web/webmvc.html

Section Summary

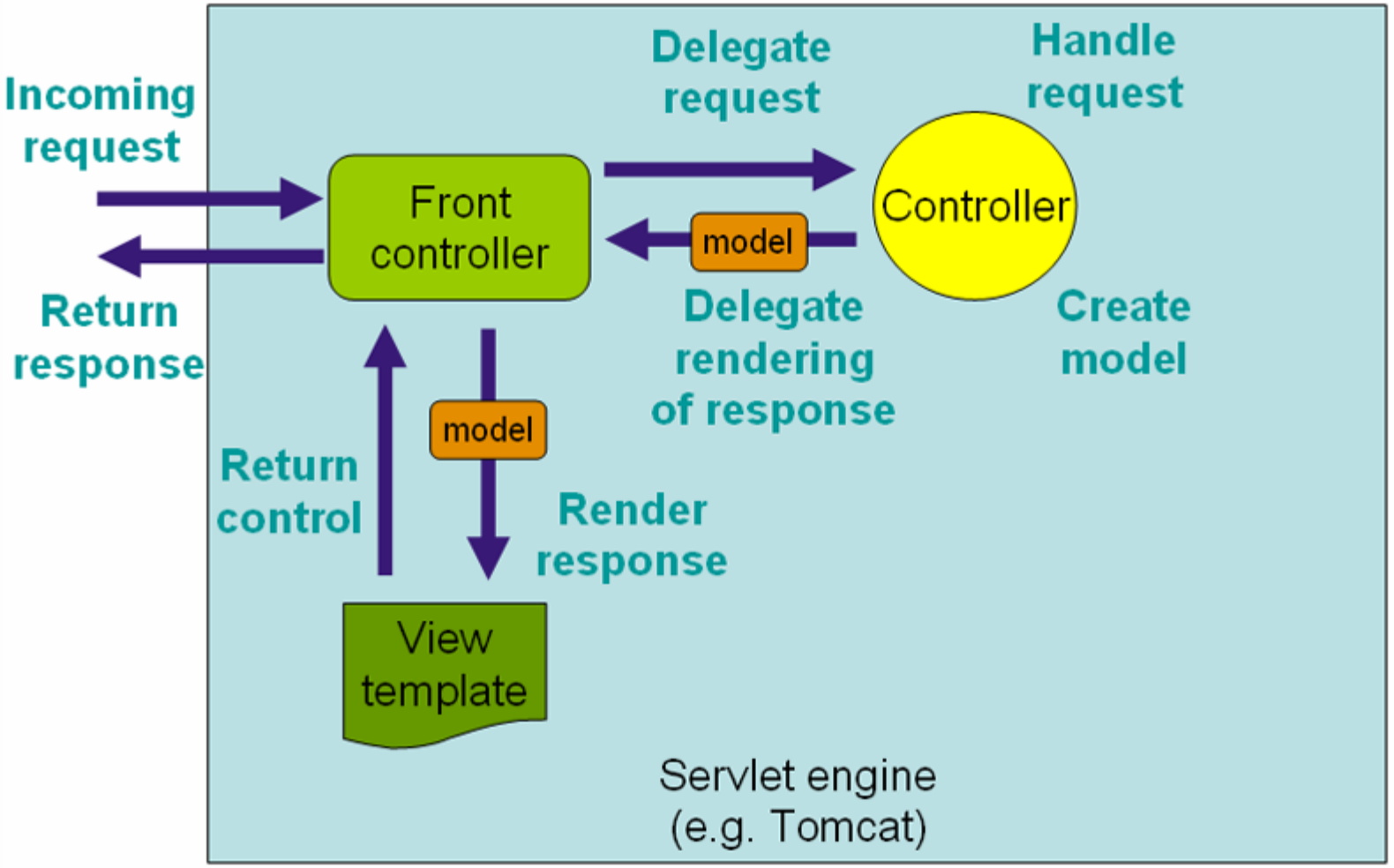
- DispatcherServlet
- Filters
- HTTP Message Conversion
- Annotated Controllers
- Functional Endpoints
- URI Links
- Asynchronous Requests
- Range Requests
- Data Binding
- CORS
- API Versioning
- Error Responses
- Web Security
- HTTP Caching
- View Technologies
- MVC Config
- HTTP/2

Architecture of Spring MVC

Core architecture (the request flow)

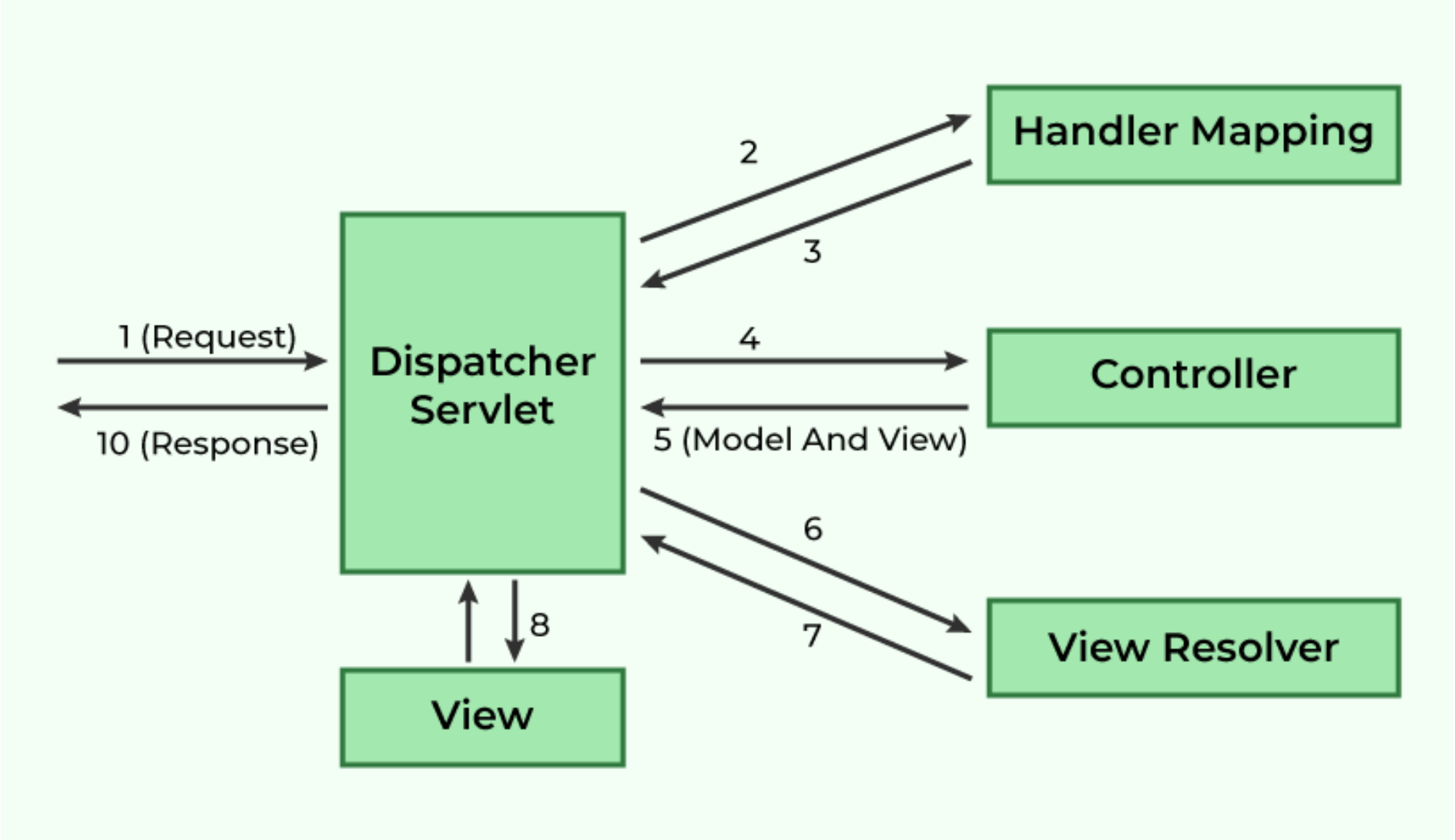
- Spring MVC relies on a Front Controller pattern.
- The central component managing this lifecycle is the DispatcherServlet
- How a request is processed
  1. The client sends an HTTP request, which is intercepted by the DispatcherServlet.
  2. Handler Mapping: the DispatcherServlet consults HandlerMapping to find the appropriate controller method that matches the request URL.
  3. Controller Execution: the chosen controller processes the request (often communicating with service and database layers).
  4. Returning Model & View: the controller returns a ModelAndView object (containing the data model and the logical name of the view) back to the DispatcherServlet.
  5. View Resolution: the DispatcherServlet asks a ViewResolver to map the logical view name to an actual UI template (like a JSP or Thymeleaf file).
  6. Rendering: the view renders the final HTML page using the model data and sends it back to the client.

Architecture overview



More reference about the Front Controller pattern
https://martinfowler.com/eaaCatalog/frontController.html

Architecture detail



Controller

- Spring MVC provides an annotation-based programming model
  - @Controller
  - @RestController

```
@Controller
public class HelloController {

    @GetMapping("/hello")
    public String handle(Model model) {
        model.addAttribute("message", "Hello World!");
        return "index";
    }
}
```

Method accepts a Model

Method returns a view name as a String

Key annotations

- Instead of managing extensive XML files, modern Spring MVC applications rely heavily on Java annotations to route and process traffic
  - @Controller
  - @RestController
  - @RequestMapping
    - @GetMapping
    - @PostMapping
    - @PutMapping
    - @DeleteMapping
    - @PatchMapping
  - @RequestParam
  - @PathVariable

Controller’s responsibility

- Receive and convert input from HTTP requests.
- Call Service classes
- Feed data into the model.
- Choose a view or a redirect.
- Do not access the database directly.

Mapping request to Controller

- Use the @RequestMapping annotation to map requests to controller methods.
- Can use annotation in class level or method level
- URI patterns: link

```
@RestController
@RequestMapping("/persons")
class PersonController {

    @GetMapping("/{id}")
    public Person getPerson(@PathVariable Long id) {
        // ...
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public void add(@RequestBody Person person) {
        // ...
    }
}
```



Old way: Servlet/JSP

Problem	Description
Boilerplate code	Each servlet must extend HttpServlet, override doGet/doPost
Complex configuration	Each servlet must be declared in web.xml.
Manual mapping	Manually parse URLs and parameters, and handle forwarding/redirecting.
Hard to test	Servlets are container-dependent and difficult to unit test.
Manual form handling	request.getParameter() for each field
Not apply Dependency Injection	Self-initializing dependencies are difficult to manage.
Difficult to scale	To add interceptors or validation, you have to write the code manually.



Enterprise level systems

Comparison

Servlet/JSP	Spring MVC/Thymeleaf
HttpServlet	@Controller
doGet / doPost	@GetMapping / @PostMapping
request.setAttribute	model.addAttribute
RequestDispatcher.forward	Returns the logical view name.
JSP + EL/JSTL	Thymeleaf expressions

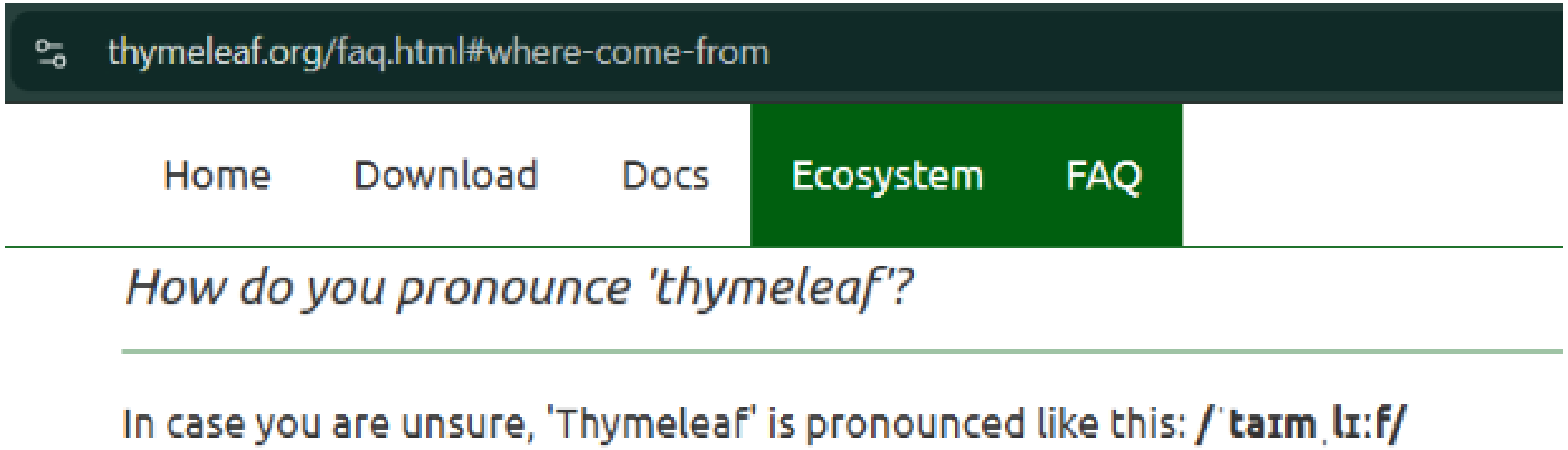
Thymeleaf

Thymeleaf

- **Server-side** Java template engine.
- Thymeleaf is not a Spring Framework project.
- Get HTML template and data in Model to create complete HTML.
- The template is still close to standard HTML, can be opened to see the structure.
- In Spring MVC, Thymeleaf takes care of the View layer.
- Do not handle business logic in the template.
- Official website: https://www.thymeleaf.org/



Thymeleaf pronunciation



Thymeleaf is pronounced as (time-leef) or /ˈtaɪmˌliːf/

https://www.thymeleaf.org/faq.html#where-come-from

Key features

- Attribute-based syntax: uses **th:*** attributes (like th:text or th:if) to inject dynamic data into standard HTML tags.

- Offline prototyping: frontend designers can view and edit templates directly in a browser without running a backend server.

- Multiple template modes: Supports HTML, XML, Text, JavaScript, CSS, and Raw processing.

Example

```
1 <h1 th:text="${product.name}">Product name</h1>
2
3 <li th:each="product : ${products}">
4   <a th:href="@{/products/{id}(id=${product.id})}" th:text="${product.name}">
5     Product name
6   </a>
7 </li>
8
9 <p th:if="${#lists.isEmpty(products)}">
10     There is no product.
11 </p>
```

Summary

- The architecture and request flow
- The DispatcherServlet coordinates the entire request lifecycle.
- The controller receives HTTP input, calls the service, creates the model, and selects the view.
- Thymeleaf render Model to HTML.

Demo

Demo

- Goal: create and debug a Spring MVC project in Visual Studio Code

- Extensions
  - Extension Pack for Java
  - Spring Boot Extension Pack

- Steps
  - Open the Command Palette using Ctrl+Shift+P
  - Type and select Spring Initializr: Create a Maven Project
  - Specify Spring Boot version: Choose the latest stable version.
  - Language: Java.
  - Group ID: e.g., com.example
  - Artifact ID: e.g., mvc-demo
  - Packaging type: Jar
  - Java version: Select your installed version (e.g., 17 or 25).
  - Dependencies: Search for and select Spring Web (This includes Spring MVC, RESTful capabilities, and an embedded Tomcat server)

Add Thymeleaf template engine

- Open your pom.xml file

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

Add a simple Controller

- Navigate to src/main/java/com/example/demo/ in your explorer
- Create a new file named HomeController.java

```
@Controller
public class HomeController {
  @GetMapping("/")
  public String index(Model model) {
    model.addAttribute("message", "Hello World");

    return "index";
  }
}
```

Add the index.html file

- Adding the src/main/resources/templates/index.html file

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Spring MVC demo</title>
</head>
<body>
    <h1 th:text="${message}">Message</h1>
</body>
</html>
```

Auto compile code

- This is the industry-standard approach.
- The library automatically monitors the entire project
- Whenever you save a file, it instantly restarts the server.
- Open your pom.xml file

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <optional>true</optional>
</dependency>
```

Q&A – Thank you